# GRADESENSE

- **Explainable AI for Paper Grade Transitions**
- **Early prediction of off-spec risk**
- **Safe, explainable operator guidance**
- **Honeywell Hackathon | GradeSense Team**

# PROBLEM STATEMENT

**<u>Paper grade transitions are difficult to stabilize because quality depends on interacting variables over time.</u>**

- Off-spec behavior is often detected after material has already been produced.
- Sensor trends, confidence, alerts, and historical outcomes are fragmented.
- Black-box predictions do not explain why risk changed or which action is safe.
- GradeSense must predict early, explain clearly, recommend safely, and keep operators in control.

# TECHNICAL ARCHITECTURE

- React + TypeScript dashboard -> FastAPI REST and WebSocket layer -> domain services
- Snapshot ML + direct-horizon forecasting -> explainability + constraint-aware recommendation engine
- SQLAlchemy + PostgreSQL/SQLite persist predictions, alerts, decisions, outcomes, audit, and model registry
- Docker, Nginx, Alembic, health checks, and read-only model artifacts support deployment

# DASHBOARD + AI PIPELINE

- Dashboard: live trends, off-spec risk, alerts, recommendations, analytics, history, and system health
- AI pipeline: validate -> engineer features -> predict/forecast -> explain -> constrain -> simulate -> rank
- WebSocket events keep sensor updates, predictions, drift, alerts, and recommendations synchronized

# BUSINESS IMPACT + INNOVATION

- Earlier risk visibility supports intervention before quality drift becomes avoidable waste.
  - Direct-horizon forecasting projects the transition without recursive error accumulation.
  - Recommendations are constraint-checked and simulated against a persisted baseline.
  - Explanations, operator decisions, and observed outcomes create an auditable learning loop.

# RESEARCH AND REFERENCES

- GradeSense repository: README.md | docs/SYSTEM_ARCHITECTURE.md | docs/MODEL_OVERVIEW.md | docs/API_REFERENCE.md | docs/DEPLOYMENT.md | backend/app | frontend/src